

Project's Report

1. Summary of the Source

The selected source is the blog post “Building an Intrusion Detection System on UNSW-NB15 Dataset Based on Machine Learning Algorithm” by Subrata Maji, together with the accompanying public GitHub repository SubrataMaji/IDS-UNSW-NB15. The source presents a machine-learning-based approach for building an Intrusion Detection System (IDS) using the UNSW-NB15 network intrusion detection dataset.

Problem Being Addressed

The problem addressed by the source is binary network intrusion detection. Given a network traffic record, the task is to classify it as either normal traffic or attack traffic. In the dataset, this is represented by the binary label column, where 0 represents non-attack traffic and 1 represents attack traffic.

This is a supervised classification problem. The model receives network-flow features such as protocol, service, connection state, packet statistics, byte counts, traffic rates, and other traffic-related attributes, and learns to distinguish between benign and malicious network behavior.

Why the Problem Is Important

Intrusion detection is an important cybersecurity task because organizations need to identify suspicious or malicious activity in their networks. Network attacks such as denial of service, brute-force attempts, exploitation attempts, reconnaissance, and other malicious behaviors can damage systems, disrupt services, or expose sensitive information.

Traditional intrusion detection systems may rely on fixed signatures or manually defined rules. These approaches can be effective for known attacks, but they may struggle to detect new or changing attack patterns. Machine learning is proposed as a way to learn patterns from data and improve the detection of suspicious network behavior, especially in cases where attacks differ from previously known signatures.

The problem is also practically difficult because IDS systems must balance two competing goals. They should detect as many attacks as possible, but they should also avoid producing too many false alarms. A false negative means an attack was missed, while a false positive means normal traffic was incorrectly flagged as malicious. Both types of errors have important cybersecurity consequences.

Proposed Solution

The source proposes building an IDS using machine-learning classification models trained on the UNSW-NB15 dataset. The author’s approach begins with reading and cleaning the dataset, followed by exploratory data analysis. The source then performs feature engineering and preprocessing, including handling feature relationships, preparing categorical and numerical features, standardizing data, and applying one-hot encoding.

After preprocessing, the author trains and compares several machine-learning models for the binary classification task. The goal is to identify the best-performing model for distinguishing attack traffic from normal traffic. The source emphasizes not only high detection performance, but also reducing the false alarm rate, which is especially important in intrusion detection.

Dataset Used

The dataset used in the source is UNSW-NB15, a benchmark dataset for network intrusion detection. It contains normal network traffic and multiple categories of attack traffic. The dataset includes attack types such as Fuzzers, Analysis, Backdoors, Denial of Service, Exploits, Generic, Reconnaissance, Shellcode, and Worms.

For the final reproduction, I used the full UNSW-NB15 CSV dataset consisting of the four original CSV files: UNSW-NB15_1.csv, UNSW-NB15_2.csv, UNSW-NB15_3.csv, and UNSW-NB15_4.csv, together with the feature-description file. This dataset contains approximately 2.54 million records and 49 columns before preprocessing, which matches the scale and feature count described by the source much better than the smaller predefined train/test version used in an earlier exploratory attempt.

Using the full dataset was important because the original blog reports experiments on a dataset of the same general scale. Therefore, the final reproduction is based on the full dataset rather than on the smaller Parquet train/test files.

Model or Methodology Employed

The methodology used in the source follows a standard supervised machine-learning pipeline for tabular cybersecurity data. First, the data is loaded and cleaned. Then exploratory data analysis is used to understand the distribution of the target variable, feature behavior, correlations, and attack categories. Feature engineering is then applied to prepare the data for machine-learning models.

The author evaluates several classification models, including Logistic Regression, Linear Support Vector Machine, Decision Tree, Random Forest, XGBoost or Gradient Boosting, and ensemble methods. The models are compared using metrics such as AUC, F1 score, accuracy, and false alarm rate. The source concludes that tree-based ensemble methods, especially Random Forest using selected important features, provide strong performance for this intrusion detection task.

Overall, the source presents the IDS problem as a binary classification task and proposes a machine-learning workflow consisting of data cleaning, EDA, feature engineering, model training, and model comparison on the UNSW-NB15 dataset.

2. Critical Evaluation

Main Claims Made by the Author

The author's main claim is that machine-learning models can be used to build an effective Intrusion Detection System using the UNSW-NB15 dataset. The source frames the problem as a binary classification task, where each network traffic record is classified as either normal traffic or attack traffic.

The author also claims that proper data preprocessing, feature engineering, and feature selection are important for improving intrusion detection performance. The workflow presented in the source includes dataset cleaning, exploratory data analysis, preprocessing, model training, and model comparison.

Another central claim is that tree-based ensemble models, especially Random Forest with selected important features, achieve the best performance among the tested models. The author supports this claim using metrics such as F1 score, AUC, and false alarm rate.

Whether the Claims Are Supported by the Presented Evidence

The author's general claim that machine-learning models can distinguish between normal and attack traffic is reasonably supported. The source presents several trained models and compares their performance using standard classification metrics. The reported results suggest that the UNSW-NB15 dataset contains useful patterns for binary intrusion detection.

My final reproduction also supports this claim more strongly than the initial attempt with the smaller dataset. After switching to the full four-file CSV version of UNSW-NB15, the results became close to the original blog's reported results. This suggests that the earlier differences were mainly caused by using a reduced dataset version, not by a fundamental failure of the author's approach.

However, the evidence is still not sufficient to fully support broad claims about real-world intrusion detection performance. The source mainly evaluates binary classification performance, but does not

deeply analyze whether the model performs well across different attack categories. A high overall F1 score can hide poor detection of rare or difficult attack types. Therefore, while the presented evidence supports the claim that the model works on this benchmark dataset, it does not fully prove that the model would be reliable in a real operational IDS environment.

Whether the Evaluation Methodology Is Appropriate

The evaluation methodology is mostly appropriate for a benchmark binary classification experiment. The use of a train/test split, several machine-learning models, and metrics such as F1 score, AUC, and false alarm rate is reasonable for evaluating whether machine-learning models can separate normal and attack records in UNSW-NB15.

There are still some limitations. First, the source relies heavily on aggregate binary classification metrics. These metrics are useful, but they do not show which attack categories are missed. In cybersecurity, this is important because missing rare attacks may be more serious than making a few mistakes on common attack types.

Second, the source does not sufficiently discuss the trade-off between false positives and false negatives. A false positive means that normal traffic is incorrectly flagged as malicious, which can overload security analysts. A false negative means that malicious traffic is classified as normal, which may allow an attack to pass undetected. Since these two errors have different security consequences, the evaluation should discuss them explicitly.

Third, the train/test split is useful for comparing models, but it does not fully test temporal or out-of-distribution generalization. Network traffic can change over time, and an IDS that performs well on a benchmark split may not necessarily perform equally well on traffic from another network, another time period, or a different attack distribution.

Possible Weaknesses or Limitations

The main reproducibility limitation found in the initial attempt was resolved by using the full original CSV dataset instead of the smaller Parquet train/test version. With the full dataset, the number of columns and the dataset size aligned much more closely with the source, and the reproduced results were close to the original blog's results.

However, some reproducibility limitations remain. The dataset is not included directly in the GitHub repository, so a reproducer must manually obtain it from an external source and place the files in the expected structure. This is reasonable because the dataset is large, but it makes exact reproduction more fragile unless the README gives precise download and execution instructions.

Another limitation is that the source does not fully document all preprocessing and feature-selection decisions in a single end-to-end script. If feature selection, cleaning, encoding, or column removal is performed before modeling, these steps should be clearly described so that the experiment can be reproduced exactly.

A further weakness is the focus on binary classification. Binary attack detection is useful, but it simplifies the intrusion detection problem. In practice, it is also important to know which types of attacks are detected well and which are missed. Without attack-category-level error analysis, the model's strengths and weaknesses are not fully understood.

Finally, the evaluation does not fully address real-world deployment issues. The UNSW-NB15 dataset is useful as a benchmark, but real network environments may have different traffic distributions, different services, and changing attack patterns. Therefore, strong performance on this dataset does not automatically imply strong real-world IDS performance.

Whether the Conclusions Are Justified

The author's conclusions are largely justified for the benchmark setting. My final reproduction using the full CSV dataset produced results that were close to the original blog's results, supporting the conclusion that machine-learning models, including Logistic Regression and Decision Tree, can achieve strong binary classification performance on UNSW-NB15. However, my executed reproduction did not include Random Forest, so it does not directly verify the author's specific claim that Random Forest is the best-performing model.

My findings therefore do not contradict the author's central conclusion. Instead, they refine its scope. The results support the claim that the proposed machine-learning workflow works well on the UNSW-NB15 benchmark, but they do not prove that the model is robust enough for real-world deployment without further validation.

A stronger conclusion would require clearer end-to-end reproducibility instructions, exact feature-selection documentation, additional robustness checks, and detailed error analysis by attack category. The source is useful as a tutorial and as an example of applying machine learning to intrusion detection, but its claims should be interpreted as benchmark evidence rather than complete proof of a production-ready IDS.

3. Feature Engineering Analysis

Whether Feature Engineering Was Performed

Feature engineering was performed both in the original source and in my reproduction. The source includes a preprocessing and feature-engineering stage before model training, which is appropriate because the UNSW-NB15 dataset contains a mixture of numerical traffic features and categorical network features.

In my reproduction, feature engineering was necessary because the raw feature table could not be passed directly into all machine-learning models. Some columns were categorical, some numerical features had very different value ranges, and some columns were not suitable as model inputs. Therefore, I applied preprocessing steps to prepare the dataset for binary intrusion detection.

Features Used

The prediction target was the binary label column, where 0 represents normal traffic and 1 represents attack traffic.

The input features included numerical network-traffic attributes and categorical protocol-related attributes. Examples of categorical features included proto, service, and state. These describe properties of the network connection and are meaningful for intrusion detection because different attacks may use different protocols, services, or connection states.

The attack_cat column was not used as an input feature because it directly describes the attack category. Since the target was binary attack detection, including attack_cat would leak information about the label and make the classification task unrealistically easy. Identifier-like fields were also treated carefully because they can lead to overfitting to dataset-specific records rather than learning general network behavior.

Transformations, Normalization, Encoding, and Feature Selection

Removing Irrelevant and Leakage Columns

The first feature-engineering step was removing columns that should not be used for prediction. The attack_cat column was removed because it represents the attack type and would cause target leakage

when predicting the binary label. Identifier-like columns were also removed or avoided when they did not represent generalizable traffic behavior.

This step is important methodologically because leakage can produce overly optimistic results. A model that uses `attack_cat` to predict whether a record is an attack is not learning general network behavior; it is using information that would not be available in a realistic binary IDS setting.

Encoding Categorical Features

Categorical features such as `proto`, `service`, and `state` were encoded using one-hot encoding. One-hot encoding was selected because these features are nominal categories, meaning they do not have a natural numerical order. For example, one protocol is not “larger” or “smaller” than another protocol.

Mathematically, one-hot encoding represents each category as a separate binary indicator. This prevents the model from incorrectly interpreting category labels as ordered numbers. From a cybersecurity perspective, this is meaningful because different protocols and services may be associated with different types of traffic behavior or attack patterns.

Standardization of Numerical Features

Numerical features were standardized using standard scaling for models that are sensitive to feature scale, such as Logistic Regression. This transformation subtracts the mean and divides by the standard deviation, producing features with approximately zero mean and unit variance.

Standardization is especially important for linear models because features with large numerical ranges can dominate the optimization process. Network traffic features often have very different scales, for example packet counts, byte counts, durations, and rates. Scaling helps ensure that the model does not give excessive importance to a feature only because its values are numerically larger.

Decision Tree models are less sensitive to feature scaling. Therefore, using a model-specific preprocessing pipeline is also reasonable: scaling is important for Logistic Regression, while tree models can often use unscaled numeric features.

Logarithmic Transformation

Some non-negative numerical features in network traffic data are highly skewed. This is common in cybersecurity datasets because most traffic records may have small values, while a small number of records may contain very large byte counts, packet counts, or durations.

For skewed non-negative features, a $\log_{1p}$ transformation was considered or applied. The $\log_{1p}$ transformation computes $\log(1 + x)$, which reduces the influence of extremely large values while still preserving zero values.

This transformation is meaningful mathematically because it compresses long-tailed distributions. It is meaningful from a cybersecurity perspective because extreme traffic values may represent unusual behavior, but they should not necessarily dominate the entire model. The transformation can make numerical features more stable for linear models and improve the quality of the decision boundary.

Feature Selection

Feature selection was considered as part of the feature-engineering process. The original source emphasizes selected important features, especially for the Random Forest model. In my reproduction, I did not execute Random Forest, so Random-Forest-based feature importance was not used as part of the final experimental results.

Feature selection can improve interpretability and reduce model complexity. However, it must be performed carefully. If feature selection is done using information from the test set, it can cause data leakage. Therefore, feature selection should be performed only within the training data or inside a cross-validation pipeline.

In the final report, the exact feature-selection decision should be described according to the final notebook results. If all features were kept, this should be stated. If selected important features were used, the selection method and the resulting feature list should be documented.

Transformations Not Applied

Some transformations were not used. For example, Box-Cox transformation was not applied because it requires strictly positive values and is less convenient for traffic features that may contain zeros. Target encoding was also not used because it can introduce leakage risk if not handled carefully, and one-hot encoding was sufficient for the main categorical features. Dimensionality reduction methods such as PCA were not used in the main experiment because they reduce interpretability, and interpretability is important in cybersecurity analysis.

Redundancy in the System

There may be redundancy in the feature set because network traffic datasets often contain related measurements. For example, packet counts, byte counts, rates, and duration-based features may be strongly correlated with each other.

Redundancy can be spotted using correlation analysis between numerical features. In my notebook, I used Spearman correlation because many network traffic features are skewed and may not follow a normal distribution. Spearman correlation is based on ranks, so it is more suitable for detecting monotonic relationships in non-normal data than Pearson correlation.

Highly correlated feature pairs suggest that two features may contain similar information. If many features are redundant, the model may become more complex than necessary, and interpretation may become harder. To tackle this issue, one possible approach is to remove one feature from each highly correlated pair. Another approach is to use model-based feature importance or regularization. For this project, redundancy analysis was mainly used to understand the feature space and discuss possible improvements.

Whether the Feature Engineering Process Was Meaningful

The feature-engineering process was meaningful because it addressed real problems in the dataset. The dataset contained categorical features, numerical features with different scales, skewed distributions, and columns that could cause leakage. Without preprocessing, some models would not work correctly, and others could produce misleading results.

Mathematically, one-hot encoding allowed categorical variables to be represented without imposing a false ordering. Standardization made numerical features comparable in scale for linear models. Logarithmic transformation reduced the effect of extreme values in skewed features. Correlation analysis helped identify possible redundancy.

From a cybersecurity perspective, the feature engineering was also meaningful. Protocol, service, and connection state are directly related to network behavior, so preserving them through encoding is important. Removing `attack_cat` was essential because a real IDS would not already know the attack category before making a prediction. Analyzing redundancy is also useful because IDS models should ideally rely on robust and meaningful traffic patterns rather than duplicated or unstable signals.

Overall, the feature-engineering process improved the methodological quality of the experiment by reducing leakage risk, preparing mixed feature types for machine learning, and making the model comparison more reliable.

Additional Features That Could Potentially Improve Performance

Additional features could potentially improve the IDS model, especially if raw packet-level data or stronger temporal information were available.

First, temporal features could be useful. Network attacks often occur in bursts or sequences, so features based on time windows, connection frequency, or recent activity could improve detection. For example, the number of connections from the same source within a short time window may help identify scanning or denial-of-service behavior.

Second, source-destination relationship features could be useful. Aggregated features based on source IP, destination IP, port pairs, or repeated communication patterns could help detect unusual behavior. These features should be designed carefully because direct use of identifiers may overfit to dataset-specific hosts.

Third, attack-category-aware analysis could improve evaluation and possibly modeling. Even if the main task is binary classification, understanding which attack categories are difficult to detect can guide the creation of better features.

Fourth, domain-specific ratio features may improve performance. Examples include bytes per packet, packets per second, source-to-destination byte ratios, and duration-normalized traffic rates. Such features may capture behavioral patterns more directly than raw counts.

Finally, graph-based features could be useful for cybersecurity analysis. Network communication can be represented as a graph of hosts and connections. Features such as node degree, connection centrality, or repeated communication patterns may help detect suspicious behavior that is not obvious from individual records alone.

4. Reproducibility Analysis

Whether the Code Can Be Executed Successfully

The original GitHub repository is publicly available and contains several notebooks that follow the workflow described in the blog post. The repository includes notebooks for dataset cleaning, exploratory data analysis, preprocessing, feature engineering, model training, and prediction. This makes the project substantially reproducible because the main stages of the author's workflow are visible.

In my reproduction, I was able to execute my own notebook successfully using the full UNSW-NB15 CSV dataset. The earlier problem caused by using a smaller Parquet train/test version was resolved by downloading and using the four original CSV files. After this change, the dataset size and number of columns were much closer to the source, and the results were close to the original blog's results.

I did not reproduce the original repository line by line. Instead, I used the author's project as the source being evaluated and implemented a controlled reproduction using the same dataset family, the full CSV data, and a similar machine-learning workflow.

Whether All Required Files and Dependencies Are Available

The GitHub repository contains the author's notebooks, but the dataset itself is not included in the repository, which is understandable because the full dataset is large. The reproducer must download the UNSW-NB15 dataset separately and place the four CSV files and the feature-description file in the expected location.

In the final reproduction, the dataset consisted of the full UNSW-NB15 CSV files rather than the smaller Parquet files used in the initial attempt:

- UNSW-NB15_1.csv
- UNSW-NB15_2.csv
- UNSW-NB15_3.csv
- UNSW-NB15_4.csv
- UNSW-NB15_features.csv or an equivalent feature-description file

The required Python dependencies were standard data science and machine-learning libraries, including pandas, numpy, matplotlib, seaborn, scikit-learn, and optionally xgboost. These dependencies are available in Google Colab or can be installed easily. Therefore, the dependency requirements are manageable.

Whether Hidden Preprocessing Steps Exist

After switching to the full CSV dataset, the largest dataset-version mismatch was resolved. However, some ambiguity remains regarding preprocessing and feature selection. The source describes a sequence of cleaning, EDA, preprocessing, and feature-selection notebooks, but exact reproduction still depends on following the same column handling, encoding, feature selection, and random split decisions.

This matters because preprocessing decisions can strongly affect model performance. Removing columns, cleaning categories, selecting features, or fitting transformations outside the training split can all influence the final results. If these steps are not fully documented in a single end-to-end reproducible pipeline, it becomes more difficult to determine whether performance differences are caused by the model itself or by differences in data preparation.

A possible hidden preprocessing issue is feature selection. The source emphasizes selected important features, especially for the Random Forest model. For exact reproducibility, the process for selecting these features should be clearly specified and applied only on the training data. If feature selection is performed using information from the full dataset or test set, it can cause data leakage and overly optimistic results. This concern refers to the author's workflow; my executed reproduction did not include Random Forest.

Overall Reproducibility of the Work

Overall, the work is reasonably reproducible as a tutorial-style machine-learning project, especially after using the full UNSW-NB15 CSV dataset. The general workflow can be reproduced: loading the dataset, assigning feature names, cleaning and preprocessing the features, training machine-learning models, and evaluating binary intrusion detection performance. The public GitHub repository is useful because it exposes the author's general methodology.

However, exact reproduction is still not perfect. The dataset must be downloaded separately, local file paths must be adapted, and some preprocessing and feature-selection choices are spread across notebooks rather than packaged as one fully automated pipeline. A more reproducible project would include exact dataset links, file checksums or filenames, dependency versions, fixed random seeds, and a single executable pipeline from raw data to final results.

Compared with the initial reproduction attempt using the smaller Parquet dataset, the final full-dataset reproduction is much stronger. The previous concern about a 36-column dataset no longer applies to the final experiment because the full four-file CSV dataset was used.

5. Experimental Results

Experiments Performed

I performed a supervised binary classification experiment on the full UNSW-NB15 dataset. The goal was to classify each network traffic record as either normal traffic or attack traffic using the binary label column.

The main experiment used the full original CSV dataset consisting of four files. After loading and combining the files, I created a train/test split similar in scale to the split used in the source. The training set was used for fitting the models, and the test set was used for final evaluation.

In addition to the main reproduction experiment, I also used additional evaluation and error-analysis procedures to better understand the results. These included confusion matrices, false-positive and false-negative analysis, and metric comparisons beyond the metrics emphasized in the original blog.

Modifications Introduced

Several modifications were introduced compared with a direct line-by-line execution of the original source.

First, I loaded the full UNSW-NB15 CSV dataset directly and assigned feature names using the feature-description file. This replaced the earlier exploratory attempt that used smaller Parquet train/test files. The full CSV version is closer to the dataset described in the source.

Second, I removed columns that could cause leakage or were not meaningful as model inputs. The `attack_cat` column was removed because it describes the attack category and would leak information when predicting the binary label. Identifier-like columns were also treated carefully because they may not generalize beyond the dataset.

Third, I implemented preprocessing using scikit-learn pipelines. Categorical features were encoded using one-hot encoding, numerical features were standardized where appropriate, and skewed non-negative numerical features were handled using logarithmic transformation where appropriate. Using a pipeline made preprocessing more systematic and reduced the risk of accidental data leakage.

Fourth, I added additional evaluation metrics beyond simple accuracy. Since intrusion detection has different costs for false positives and false negatives, I evaluated the models using precision, recall, F1 score, F2 score, MCC, ROC-AUC, average precision, false positive rate, false negative rate, and confusion matrices.

Models Trained

I trained models from the same general families used in the original source. The main models included:

1. Logistic Regression
2. Decision Tree

Logistic Regression was used as a simple and interpretable baseline model. It tests whether the engineered features can separate normal and attack traffic using a mostly linear decision boundary.

Decision Tree was used as an interpretable nonlinear model. It can capture threshold-based patterns in network traffic features and provides a useful comparison to the linear Logistic Regression baseline.

Random Forest was discussed in the original source, but it was not included in my final executed reproduction, so this report does not claim to reproduce the author's Random Forest result.

Evaluation Metrics

The models were evaluated using several metrics.

Accuracy measures the overall fraction of correct predictions, but it is not sufficient on its own because intrusion detection datasets may be imbalanced.

Precision measures how many predicted attacks were actually attacks. In an IDS context, low precision means many false alarms.

Recall measures how many real attacks were detected. In an IDS context, low recall is dangerous because attacks may pass undetected.

F1 score combines precision and recall equally. It is useful when both false positives and false negatives matter.

F2 score gives more weight to recall than precision. This is useful in intrusion detection because missing an attack is often more serious than raising an extra alert.

Matthews Correlation Coefficient is useful because it considers all parts of the confusion matrix and is more informative than accuracy when classes are imbalanced.

ROC-AUC measures the model's ability to rank attacks above normal traffic across different thresholds.

Average precision summarizes the precision-recall curve and is useful when class imbalance is important. The confusion matrix was also used to directly analyze true positives, true negatives, false positives, and false negatives.

Obtained Results

The following table should be filled with the final values from the notebook. The key qualitative result is that after switching to the full UNSW-NB15 CSV dataset, the obtained results were close to the original blog's reported results.

Model	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC	False Positive Rate	False Negative Rate
Logistic Regression	0.985804	0.773246	0.999799	0.872048	0.944456	0.872670	0.998774	0.014907	0.000201
Decision Tree	0.986261	0.779103	0.999398	0.875607	0.945906	0.876002	0.996607	0.014407	0.000602

Based on the final results, the best-performing model was Decision Tree. It achieved the strongest overall balance between attack detection and false-alarm control, with F1 of 0.875.

The results support the general claim that machine-learning models can classify normal and attack traffic in the UNSW-NB15 dataset. In contrast to the earlier reduced-dataset attempt, the full-dataset reproduction produced results that were close to the original blog's results. Therefore, the final experiment supports the source's main benchmark conclusion more strongly.

Interpretation of the Results

The experimental results show that the full UNSW-NB15 dataset contains useful patterns for binary intrusion detection. The models were able to learn differences between normal and attack traffic using the available network features.

The main correction compared with the earlier report draft is that the final reproduction no longer depends on the smaller Parquet train/test dataset. Because the full four-file CSV dataset was used, the previous criticism about mismatched dataset size and 36 columns is no longer part of the final conclusion.

However, the results still highlight the importance of careful evaluation. High accuracy or high F1 score does not automatically mean that the model is suitable for real-world deployment. In intrusion detection, false negatives are especially important because they represent attacks that were missed. False positives are also important because they create unnecessary alerts and may overload security analysts.

Therefore, the most meaningful interpretation of the results comes from combining aggregate metrics with confusion matrices, false positive and false negative rates, and error analysis by attack category. The results support the use of machine learning as a useful approach for IDS modeling, but they do not fully prove that the model would generalize to real-world network environments without further validation.

6. Conclusions

This project critically evaluated the blog post "Building an Intrusion Detection System on UNSW-NB15 Dataset Based on Machine Learning Algorithm" and reproduced its main machine-learning workflow using the full UNSW-NB15 dataset. The original source presents intrusion detection as a binary classification problem, where network traffic records are classified as either normal or attack traffic. Overall, my reproduction supports the author's main claim that supervised machine-learning models can achieve strong performance on this dataset.

The reproduction results were close to the original blog's reported behavior, especially after switching from the smaller predefined train-test dataset to the full four-file UNSW-NB15 CSV dataset. This shows that the dataset version has a major effect on reproducibility. Once the full dataset was used, the experimental setup became much more consistent with the source, and the results were more comparable to the author's claims.

The main strength of the proposed solution is that it follows a clear and practical machine-learning pipeline: data loading, preprocessing, feature engineering, model training, and evaluation. The dataset contains meaningful network-traffic features, and the executed models were able to detect attack traffic effectively. In my results, the Decision Tree achieved very high recall, showing that the model was able to identify almost all attack samples in the evaluated split. This is important in an IDS context, where false negatives represent attacks that pass undetected.

However, the project also has limitations. First, high binary classification performance does not necessarily prove that the system is ready for real-world deployment. Real network environments may have different traffic distributions, new attack types, and temporal changes that are not fully represented in the dataset. Second, binary evaluation can hide weaknesses across specific attack categories. A model may perform very well overall while still failing on rare or difficult attack types. Third, false positives and false negatives have different operational meanings in cybersecurity, so accuracy alone is not enough. Evaluation should include recall, precision, F1/F2 score, MCC, false positive rate, false negative rate, confusion matrices, and error analysis.

The feature-engineering process was meaningful because the dataset contains both categorical and numerical features. Removing leakage columns, encoding categorical variables, scaling numerical features, and analyzing redundancy all helped create a more methodologically sound experiment. These steps are important not only mathematically, but also from a cybersecurity perspective, because an IDS should learn from realistic traffic behavior rather than from leaked labels or artificial identifiers.

In conclusion, the author's main claim is mostly justified within the context of the UNSW-NB15 benchmark dataset. Machine-learning models can successfully distinguish normal and attack traffic in this setting, and the Decision Tree result shows that tree-based decision rules can be effective for this task. However, my reproduction did not execute Random Forest, so it does not directly confirm the author's specific claim that Random Forest is the best-performing model. In addition, the results should be interpreted as evidence of strong performance on this dataset and split, not as proof of reliable detection on future traffic, unseen hosts, or another network environment. A stronger future study would include attack-category-level evaluation, temporal validation, host-based or network-based splits, testing on additional datasets, and a deeper analysis of false positives and false negatives. Therefore, I would recommend this project as a useful educational and experimental IDS pipeline, but not as a complete real-world intrusion detection solution without further validation.